Inter IIT Tech Meet 15.0: Sarvam.ai Mini PS

In the Sarvam challenge, each team builds a system that listens to live Hindi speech, freely mixed with English, and speaks the English translation while the speaker is still talking.

1. The task(Points 100)

Write a report, backed by a small pipeline you build, that convinces us you understand this problem deeply. Go deep rather than wide.

Part A: Understanding the problem (Points 10)

- What is the system trying to do, and where are points won and lost?
- Walk through one session, from the first audio frame to `session_complete`. What matters most for latency?
- Which rules are most important or most easily misunderstood?
- A speaker says "पाँच लाख… नहीं नहीं, पाँच करोड़।" When can the system safely speak the amount, and how would you teach a model to wait only when it must?

In the appendix, handwritten: for "मैंने कल अपना पुराना बैंक खाता बंद करवा दिया", write the wait-3 output schedule and compute its Average Lagging.

Getting started. Read the official problem statement end to end; its rules, KPIs and FAQ answer most of these questions. The STACL paper(in reference) defines wait-k, and the SimulEval paper defines Average Lagging.

Part B: Research and ideas (Points 40)

Pick **four** of the open questions below, or propose your own. For each, search widely (papers, datasets, repositories, model cards, technical blogs) and write a short proposal: your recommendation, the evidence behind it, the main risk, and the cheapest experiment that would confirm or rule it out.

1. **Data sources.** Which Hindi and code-mixed speech datasets can we legally redistribute, how many usable hours do they hold, and how should real and synthetic audio be mixed within 200 hours?
2. **Training targets.** How literal should the English be, and how should Hindi and English sentences map, for a model that cannot take back speech?
3. **Timing the English.** How should English words be placed in time against the Hindi so the model learns when it is safe to speak? How do existing systems build such aligned pairs?
4. **Hard content.** How do we get enough amounts in lakh and crore, phone numbers, names, negations and self-corrections into the data, and check that TTS rendered them correctly?
5. **Fine-tuning blind.** The base model's architecture is revealed only after the mid-term. What can we prepare now to fine-tune well in the time left?
6. **Latency.** Which decoding-time choices reduce lag without hurting entity accuracy?

List every source you used in a table in the appendix (aim for at least 10), each with one line on what it told you. For each proposal, also tell us:

- one question that matters for us and that no source answers
- one claim, from a paper or from an AI assistant, that turned out wrong or misleading when you checked it

Finally, give us **one original idea** of your own: why it should work here, how you would test it in the first week, and when it would fail.

Getting started. Begin with the references, then follow the papers they cite and the papers that cite them, using Google Scholar or Semantic Scholar. Hugging Face dataset and model cards list licences and hours. Read the method and limitations sections, not just abstracts, and check every claim you rely on against the original source.

Part C: Designing the dataset(Points 25)

The dataset decides most of the score in this problem. Research how speech-translation and Indic speech datasets have been built, and propose how we should build ours. For every choice, compare the options you found, cite who used them and what happened, and recommend one.

- **Sources and generation.** Which existing datasets, TTS systems and text sources could we use, with their licences and usable hours? How have others generated code-mixed or entity-heavy speech? How would you split the 200-hour budget?
- **Transcripts, timings and translations.** Which tools and methods exist for verbatim transcription, forced alignment and translation of code-mixed Hindi? How accurate are they reported to be, and where is a human still needed?
- **Validation.** How have other dataset builders filtered bad audio, timings and translations, for example with ASR round-trips, alignment confidence or back-translation? Which checks would you adopt, and with what thresholds?
- **Real-world conditions.** What is known about the gap between synthetic and real speech, and which methods close it: noise, telephone and codec simulation, injected disfluencies, mixing in real recordings? How would you measure that gap?
- **Failure modes.** What has gone wrong in similar datasets, and what could make ours pass the audit but still train a worse model?

End with the pipeline you recommend, drawn as one diagram, and the three open questions you would test first.

Getting started. Sections 3 and 5.2 of the official problem statement define what each record must contain and how hours are counted. The IndicVoices and IndicVoices-R papers show how a large Indic speech dataset was collected, transcribed and cleaned, and the Hibiki paper shows how synthetic aligned training pairs can be generated.

Part D: Build and measure(Points 10)

- Record 6 clips (20 to 40 s) of natural code-mixed Hindi, with amounts in lakh or crore, a phone number, names, a negation and self-corrections.
- Transcribe them verbatim in IndicVoices Level 1 style, and produce word-level timings with any open aligner. Check 30 boundaries by hand and report the error.
- Translate each sentence literally and fluently, and write a checker that verifies amounts, numbers, names and negations in the English. Report its accuracy and where it fails.

Getting started. A phone recording in a quiet room is enough; convert it with ffmpeg or sox. The IndicVoices paper describes Level 1 transcription. The torchaudio multilingual forced-alignment tutorial works on Hindi once the transcript is romanised, and Audacity's label track lets you mark word boundaries by hand. Any open model, such as IndicTrans2, or an LLM can translate; the checker can be plain Python with regular expressions and a table of Hindi number words.

Part E: Track task (Points 15)

Choose one, matching the role you want.

- **ML:** Simulate a wait-k pipeline on your clips for several values of k, and plot entity accuracy against latency. Where should a streaming model wait?
- **Dev:** Build a WebSocket server following Section 5.1 of the official problem statement that echoes the input after a delay, with a test client and a Dockerfile. What could make it fall behind, and how would you prevent it?
- **Signal Processing:** Build a causal front-end (resampling, denoising, silence trimming) that works on 80 ms frames. Test it on noisy and telephone-band copies of your clips, and find a case where denoising hurts.

Getting started.

- **ML:** Use your Part C word timings as the Hindi timeline. Under wait-k, English word i is spoken once k + i Hindi words have been heard; give each English word a fixed duration or measure it from TTS audio. Score each run with your Part C entity checker.
- **Dev:** Python's asyncio with the websockets library, or FastAPI, is enough. Run reading and sending as separate tasks so neither blocks the other. Section 5.1 of the official problem statement gives every message format. Test with your own client first, then package with a slim Python Docker image.
- **Signal Processing:** Make the telephone-band and noisy copies with sox, ffmpeg or scipy. Compare a classical denoiser (spectral subtraction or a Wiener filter) with DeepFilterNet, and use Silero VAD for silence trimming. Process audio in a loop over 80 ms frames to keep it causal, and measure SI-SDR against your clean recording.

2. Submission

- **Report:** one PDF, at most 4 pages, or you can generate an artifact from the LLM you used
- **Code:** a repository link with your scripts, clips and a `RESULTS.md` listing every number with the command that reproduces it.
- **File name:** `<your_name>_mini_ps.pdf`.

- **Rules:** individual work. Any tools may be used, but in the interview you will run your code on a new clip and explain every part of it.

3. References

Starting points for study. Part B expects you to go well beyond them.

The problem

- Sarvam problem statement (PDF, shared with this document)
- Sarvam API documentation

Simultaneous speech translation

- Hibiki: High-Fidelity Simultaneous Speech-to-Speech Translation and its code
- Moshi: a speech-text foundation model for real-time dialogue
- StreamSpeech: Simultaneous Speech-to-Speech Translation with Multi-task Learning and its code
- Seamless: Multilingual Expressive and Streaming Speech Translation
- STACL: Simultaneous Translation with Implicit Anticipation and Controllable Latency (wait-k)
- Turning Whisper into a Real-Time Transcription System (local agreement)

Latency and evaluation

- SimulEval: An Evaluation Toolkit for Simultaneous Translation
- Length-Adaptive Average Lagging (LAAL)

Indic speech and translation

- IndicVoices (AI4Bharat, IIT Madras, Sarvam AI)
- IndicVoices-R
- IndicSUPERB and the Kathbath dataset
- IndicTrans2

Alignment and audio tools

- Scaling Speech Technology to 1,000+ Languages (MMS)
- torchaudio: forced alignment for multilingual data
- WhisperX: time-accurate speech transcription
- Montreal Forced Aligner
- Silero VAD and DeepFilterNet